LatticeMico32 UART

The LatticeMico32 UART is a universal asynchronous receiver-transmitter used to interface to RS232 serial devices. The UART is based on, and has many characteristics similar to, defacto standard 16450 UARTs. To preserve FPGA resources, the LatticeMico32 UART is not identical to the 16450, so it is not source-code-compatible. However, if you are familiar with the 16450, you will be able to implement the LatticeMico32 UART very quickly.

Features

The UART includes the following features:

- WISHBONE B.3 interface
- Based on the NS16450 UART
- Insertion or extraction of standard asynchronous communication bits (start, stop, and parity) to or from the serial data
- Holding and shifting registers, which eliminate the need for precise synchronization between the CPU (WISHBONE interface) and serial data
- A common interrupt line for all internal UART data and error events. Interrupt conditions include receiver line errors, receiver buffer available, transmit buffer empty, and detection of status flag change.
- Fully prioritized interrupt system control
- Optional modem function (not presently supported)
- Support for instantiating the UART multiple times
- Fully programmable serial interface characteristics:
  - 5-, 6-, 7-, or 8-bit characters
  - Even-, odd-, or no-parity bit generation and detection

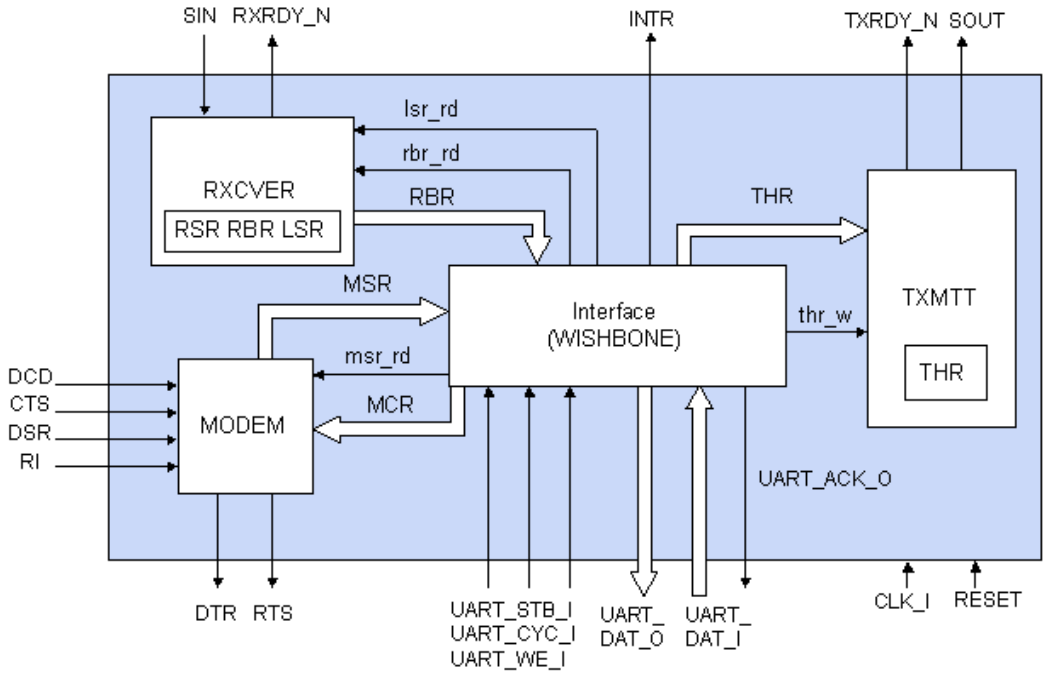
- 1-, 1.5-, or 2-stop bit generation and detection
- False-start bit detection
- Line-break generation and detection
- Interactive control signaling and status reporting capabilities

For additional details about the WISHBONE bus, refer to the *LatticeMico32 Processor Reference Manual*.

Functional Description

The LatticeMico32 UART provides an interface between the LatticeMico32 WISHBONE system bus and a RS232 serial communication channel. Figure 1 shows the major blocks implemented in the UART.

Figure 1: UART Usage Diagram



UART Clock Frequencies

The UART only has a single clock input. The UART uses the WISHBONE CLK_I input frequency to transfer data between the LatticeMico32 microprocessor and the UART control and status registers. It also uses CLK_I to generate an internal clock that is 16 times the inter-device baud rate. This clock is referred to as CLK16X. The CLK16X frequency is established when the UART is defined in the LatticeMico32 Mico System Builder (MSB) interface. It can also be changed under software control by modifying the value in the baud-rate divisor register, as described in “Baud-Rate Divisor Register” on page 16.

Receiver

The serial receiver (RXCVER) section contains an 8-bit receiver buffer register (RBR) and receiver shift register (RSR).

Since the serial frame is asynchronous to the receiving clock, a high-to-low transition of the SIN pin is treated as the start bit of a frame. However, to avoid receiving incorrect data because of SIN signal noise, false-start bit detection is implemented. The UART requires the start bit to be low at least 50 percent of the baud rate clock. The UART samples SIN for 8 CLK16X cycles, and if all eight samples are low, a start bit is detected.

Once a valid start bit is received, the data bits and the parity bit are sampled every 16 CLK16X clocks (the RS232 baud rate). If the start bit is exactly 16 CLK16X clocks long, each of the following bits will be sampled at the center of the bit itself.

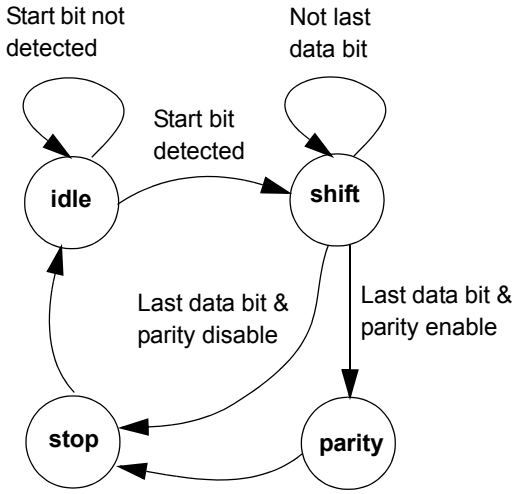
The receive logic monitors the incoming data stream and reports the state of the incoming line in the line status register (LSR). The LSR is used to indicate overrun, parity, and framing errors. It also indicates when a line break has been received.

Whenever a framing error is detected, the UART assumes that the error was due to the start bit of the following frame and tries to resynchronize it. To do this, it samples the start bit twice. If both samples of the SIN are low, the UART resynchronizes and accepts the data following the second start bit. The resynchronization does not occur for a framing error caused by a break character.

Once every bit is received, the data in the receive shift register is transferred to the receive buffer register (RBR). The data-ready bit in the LSR is set to 1 to indicate to the CPU that a byte of data has been received. The external rxrdy_*n* output is asserted (that is, logic 0) simultaneously with the data-ready bit. You can also configure the UART to generate an interrupt upon successful transfer from the RSR to the RBR.

The behavior of the receiver is controlled by the finite state machine (FSM), as shown in Figure 2.

Figure 2: Receiver State Machine



The receiver state machine includes the following states:

- idle

  When reset or after the stop state, the receiver FSM is reset to this state. When in this state, it waits for SIN to be changed from high to low and stay low for 8 CLK16X clocks to be considered a valid start bit. Once a valid start bit is detected, the FSM switches to the “shift” state.

- shift

  When the FSM is in this state, it waits 16 CLK16X clocks for each data bit to shift into the RSR. After the last data bit is shifted in, the FSM switches to the “parity” state if parity is enabled. Otherwise, it switches to the “stop” state.

- parity

  When the FSM is in this state, it waits for 16 CLK16X clocks and then samples the parity bit. Once the parity bit is sampled, the FSM switches to the “stop” state.

- stop

  Whether the stop-bit length is configured to be 1, 1.5, or 2 bits long, the FSM always waits for 16 CLK16X clocks and then samples the stop bit. As long as a logic 1 is sampled at the stop bit, the framing error flag (FE) in LSR is not set. The receiver does not check whether the stop bit is in the right length as configured. The FSM switches back to the “idle” state after sampling the stop bit.

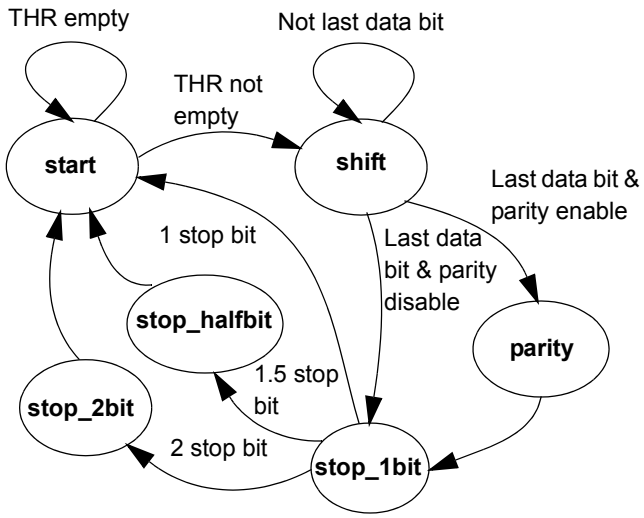
Transmitter

The serial transmitter (TXMITT) section consists of an 8-bit transmitter hold register (THR) and transmitter shift register (TSR). The UART provides two methods of indicating the status of THR: a txrdy_*n* output signal or a transmit holding register empty (THRE) flag in the line status register (LSR). When THR is empty, the txrdy_*n* pin becomes low active, and the THR empty flag in LSR is set to a logic 1. A write to the THR interferes with a transmission in progress if THRE is active or trdy_*n* is deasserted. After the data is loaded in THR, the THR empty flag in LSR is reset to logic 0, and the txrdy_*n* pin goes inactive high.

The serial data transmission is automatically enabled after the data is loaded into THR. First, a start bit (logic 0) is transmitted and the data in THR is automatically parallel-loaded to TSR. The data bits are shifted out of TSR, followed by the parity bit, if parity is enabled. Finally, the stop bit (logic 1) is generated to indicate the end of the frame. This serial data frame—start bit + data bit + parity bit + stop bit—is transmitted at the rate of 1/16 of the CLK16X frequency. After a frame is fully transmitted, another frame is transmitted immediately if THR is not empty. This automatic sequencing causes the frames to be transmitted back to back, which increases the transmission bandwidth. The SOUT pin is held high when no transmission is in progress.

The behavior of the transmitter is controlled by the finite state machine (FSM), as shown in Figure 3.

Figure 3: Transmitter State Machine



The transmitter state machine includes the following states:

- start

  When the UART is reset, the FSM transmitter is reset to this state. When in this state, the transmitter waits to assert the start bit. A start bit is asserted as soon as THR is not empty. Once a low SOUT (start bit) is asserted, the FSM switches to the “shift” state.

- shift

  When the FSM is in this state, it waits for the last (most significant) data bit to be shifted out. After the last data bit is shifted out, the FSM switches to the “parity” state if parity is enabled. Otherwise, it switches to the “stop_1bit” state.

- parity

  When the FSM is in this state, the last data bit is still in transmission. When the transmission is complete, the FSM asserts the parity bit. Once the parity bit is asserted, the FSM switches to the “stop_1bit” state.

- stop_1bit

  Whether the stop bit is configured to be 1, 1.5, or 2 bits long, the FSM always switches to this state, waits for a baud clock cycle, and then asserts the stop bits. For 1 stop bit, the FSM switches back to the “start” state and waits to assert the start bit of another frame. For 1.5 stop bits, it switches to the “stop_halfbit” state and stays there for just half a baud clock cycle before switching to the “start” state. For 2 stop bits, it switches to the “stop_2bit” state and then switches back to the “start” state. The stop bit is asserted at the time that the FSM leaves the “stop_1bit” state.

- stop_halfbit

  This state is for 5-bit data bits with a 1.5 stop bit. The FSM stays in this state for only half a baud clock cycle and then switches to the “start” state.

- stop_2bit

  When the FSM is in this state, the first stop bit is in transmission. It waits for a baud clock cycle and then asserts the second stop bit and switches to the “start” state.

Interrupt

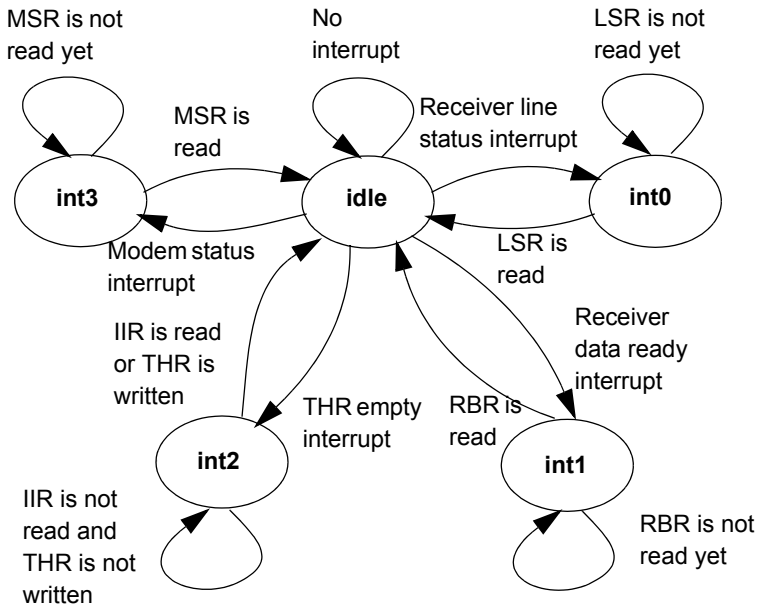
The common interrupt request pin (INTR) goes to high active when any interrupt conditions are matched and enabled by the interrupt enable register (IER), which is described in Table 5.

The UART prioritizes interrupts into four levels to minimize external software interaction, and it records these in the interrupt identification register (IIR), which is described in Table 6. The four levels of interrupt conditions in order of priority are receiver line status, received data ready, transmitter holding register empty, and modem status, which is described in Table 7.

Performing a read cycle on IIR freezes all interrupts and indicates the highest priority pending interrupt to the CPU. No other interrupts are acknowledged until the pending interrupt is serviced. Whenever the IIR is read, the current pending interrupt is cleared. Any pending lower-priority interrupt becomes visible in the IIR after the previous IIR read.

The behavior of the interrupt is controlled by the FSM, as shown in Figure 4.

Figure 4: Interrupt Behavior



The interrupt pin state machine includes the following states:

- idle

  When the UART is reset, the interrupt FSM is reset to this state. When in this state, it waits for the enabled interrupt conditions to be true, and then it switches to the interrupt state with the highest priority.

- int0

  The FSM switches to this state when the highest-priority level interrupt occurs. It stays at this state until the LSR is read.

- int1

  The FSM switches to this state when the second-priority level interrupt occurs. It stays at this state until the RBR is read.

- int2

  The FSM switches to this state when the third-priority level interrupt occurs. It stays at this state until the IIR is read or after THR is written.

- int3

  The FSM switches to this state when the fourth-priority level interrupt occurs. It stays at this state until the MSR is read.

The interrupt continues to be generated as long as the corresponding enable bit in IER is set and the corresponding interrupt condition is matched.

WISHBONE Interface

The LatticeMico32 microprocessor interfaces to the UART control and status registers by using the WISHBONE bus. The registers are accessed using classic mode data cycles. Each read and write is performed as a single cycle transfer (that is, not in burst mode).

All of the control registers are 32-bit aligned, and the SEL_I inputs are ignored. When the control registers are read or written to, only the least-significant eight bits are valid, with the exception of the baud-rate divisor register, which is 16 bits.

When the LatticeMico32 microprocessor initiates a bus cycle, it starts by asserting STB_I and CYC_I. The UART responds to the cycle with UART_ACK_O at the first rising edge following the the STB_I and CYC_I assertion. The UART_ACK_O stays active for a single CLK_I cycle, terminating the transfer and accepting or returning data.

Configuration

The following sections describe the graphical user interface (UI) parameters, the hardware description language (HDL) parameters, and the I/O ports that you can use to configure and operate the LatticeMico32 UART.

UI Parameters

Table 1 shows the UI parameters available for configuring the LatticeMico32 UART through the Mico System Builder (MSB) interface. For the data bit and the stop bit, refer to the LCR register description.

Table 1: UART UI Parameters

Dialog Box Option	Description	Allowable Values	Default Value
Base Address	Specifies the base address for configuring the UART. The minimum boundary alignment is 0x80.	0X80000000–0XFFFFFFFF	0X80000000
Block on transmit	Specifies whether the driver returns until it is able to queue data for transmission. If true, the driver does not return until it is able to queue data for transmission.	1, 0	1
Block on receive	Specifies whether the driver returns until there is data received. If true, the driver does not return until there is data received.	1, 0	1
Baud Rate	Specifies the speed in bits per second.	9600, 19200, 38400, 57600, 115200	115200
Rx Buffer Size	Specifies the size in bytes for incoming data buffer implemented by the software driver.	2-32	4
Tx Buffer Size	Specifies the size in bytes for outgoing data buffer implemented by the software driver.	2-32	4

HDL Parameters

Table 2 lists the parameters that appear in the HDL.

Table 2: UART HDL Parameters

Parameter Name	Description	Allowable Values
BAUD_RATE	Specifies the speed in bits per second.	9600, 19200, 38400, 57600, 115200

I/O Ports

Table 3 describes the input and output ports of the LatticeMico32 UART.

Table 3: UART I/O Ports

I/O Port	Active	Direction	Initial State	Description
RESET	High	I		Reset signal, active high
CLK_I	—	I		Clock signal 16 times the receiving/transmitting baud rate clock frequency
UART_ADR_I [31:0]	—	I		Address from WISHBONE interface
UART_DAT_I [31:0]	—	I		Data input from WISHBONE interface
UART_DAT_O [31:0]	—	O		Data output to WISHBONE interface
UART_SEL_I	—	I		Select input array, which indicates where the valid data is expected on a data bus
UART_STB_I	High	I		Strobe input indicating that the slave is selected
UART_CYC_I	High	I		Cycle signal indicating that a valid bus cycle is in progress
UART_WE_I	—	I		Write enable input 0 = read 1 = write
UART_BTE_I	High	I		Burst-type extension
UART_ACK_O	High	O	0	Acknowledge output, indicating the termination of a normal bus cycle
INTR	High	O	0	Interrupt request, active high, used to request service from the CPU whenever one of the following four conditions happens: ◆ Receiver line errors ◆ Receiver buffer available ◆ Transmit buffer empty ◆ Modem status flag change detected
SIN	High	I	—	RS232 serial data input
SOUT	High	O	1	RS232 serial data output

User Impact of Initial State

After reset, the transmitter idles until transmission begins.

Register Descriptions

The LatticeMico32 UART includes the registers shown in Table 4:

- Two data buffering registers, RBR and THR
- Three status registers, IIR, LSR, and MSR
- Three control registers, IER, LCR, and MCR
- Baud-rate generator, DIV

Table 4: Register Map

Register Name	Offset	31-16	15-8	7	6	5	4	3	2	1	0
Receive buffer register/ transmit holding register	0x0			Data bit 7	Data bit 6	Data bit 5	Data bit 4	Data bit 3	Data bit 2	Data bit 1	Data bit 0
Interrupt enable register	0x4			0	0	0	0	MSI	RLSI	THRI	RBRI
Interrupt identification register	0x8			0	0	0	0	0	ID1	ID0	STAT
Line control register	0xC			0	SB	SP	EPS	PEN	STB	WLS1	WLS0
Modem control register	0x10			0	0	0	0	0	0	RTS	DTR
Line status register	0x14			0	TEMT	THRE	BI	FE	PE	OE	DR
Modem status register	0x18			DCD	RI	DSR	CTS	DDCD	TERI	DDSR	DCTS
Baud-rate divisor register	0x1C		Divisor bits[15:0].								

Table 5 through Table 11 provide details about each register in the LatticeMico32 UART.

Interrupt Enable Register

The interrupt enable register provides discrete control over each of the independent interrupt sources provided by the UART. When the interrupt is masked, the INTR output is not activated when the masked interrupt source becomes active. An interrupt is masked when the corresponding register bit is set to 0.

Table 5: Interrupt Enable Register (IER, Addr = 0x04)

Register Name	Bit	Access Mode	Description
RBRI	0	Write only	Receiver buffer register interrupt (1 = enable, 0 = disable)
THRI	1	Write only	Transmitter hold register interrupt (1 = enable, 0 = disable)
RLSI	2	Write only	Receiver line status interrupt (1 = enable, 0 = disable)
MSI	3	Write only	Modem status interrupt (1 = enable, 0 = disable)

Interrupt Identification Register

The interrupt identification register holds information about which interrupt is currently active. Table 7 shows bit encoding for each interrupt source.

Table 6: Interrupt Identification Register (IIR, Addr = 0x08)

Register Name	Bit	Access Mode	Description
STAT	0	Read only	Interrupt state
ID1:0	2:1	Read only	Interrupt identification code (see Table 7)

Table 7: Four Prioritized Interrupt Levels and Sources

Interrupt Level	ID1	ID0	STAT	Source of Interrupt	Interrupt Reset Control
None	0	0	1	No interrupt pending	None
Highest	1	1	0	LSR error flags (OE/PE/FE/BI)	Reading LSR
Second	1	0	0	LSR receiver data-ready flag (DR)	Reading RBR
Third	0	1	0	LSR flag THR empty (THRE)	Reading IIR or writing THR
Fourth	0	0	0	Modem status	Reading MSR

Line Control Register

The line control register is used to define the data protocol between the interconnected serial devices. It controls the number of data bits sent, parity, the number of stop bits, stick parity, and the transmit break bit.

Table 8: Line Control Register (LCR, Addr = 0x0C)

Register Name	Bit	Access Mode	Description
WLS1:	1:0	Write only	Defines the data length: ◆ 00 – 5 bit ◆ 01 – 6 bit ◆ 10 – 7 bit ◆ 11 – 8 bit
STB	2	Write only	Stop-bit definition: ◆ 0 – 1 stop ◆ 1 – WLS1:0 = 00: 1.5 stop WLS1:0 = 01: 2 stop WLS1:0 = 10: 2 stop WLS1:0 = 11: 2 stop
PEN	3	Write only	Parity enable: ◆ 0 – Parity bit disable ◆ 1 – Parity bit enable
EPS	4	Write only	Parity even: ◆ 0 – Even parity selected ◆ 1 – Odd parity selected
SP	5	Write only	Stick parity: ◆ 0 – Stick parity disable ◆ 1 – When PEN, EPS, or SP is set (that is, 1), parity is sent or checked for 0. When PEN or SP is set, EPS is clear, parity is sent or checked for 1.
SB	6	Write only	Tx break: ◆ 0 – Disable break assertion ◆ 1 – Assert break. SOUT is driven low active (break character) as long as this bit is 1. It has no effect on the transmitter logic.

Modem Control Register

In Table 9, loopback mode for testing (bit 4) and auxiliary user-defined output Out2 (bit 3) and Out1 (bit 2) are removed, because they can be implemented with in-system programmability (ISP).

Table 9: Modem Control Register (MCR, Addr=0x10)

Register Name	Bit	Access Mode	Description
DTR	0	Write only	Controls the data-terminal-ready (DTR*n*) output. ◆ DTR=0: force DTR*n* output to a logic 1 (normal default) ◆ DTR=1: force DTR*n* output to a logic 0
RTS	1	Write only	Controls the request-to-send (RTS*n*) output. ◆ RTS=0: force RTS*n* output to a logic 1 (normal default) ◆ RTS=1: force RTS*n* output to a logic 0

Line Status Register

Table 10 provides information about the line status register.

Table 10: Line Status Register (LSR, Addr=0x14)

Register Name	Bit	Access Mode	Description
DR	0	Read only	Receiver data-ready. DR indicates status of RBR. It is set to logic 1 when RBR data is valid and is reset to logic 0 when RBR is empty. When line errors (OE/PE/FE/BI) occur, DR is also set to logic 1, and RBR is updated to reflect the data bits portion of the frame. Pin RxRDY*n* is the complement of this bit.
OE	1	Read only	Overrun error. This bit is set when the next character is transferred into RBR before the previous RBR data is read by the CPU. The previous RBR data is lost when the overrun occurs.
PE	2	Read only	Parity error. This bit is set to logic 1 only when the parity is enabled and the parity bit is not at the logic state it should be. For even parity, the parity bit should be 1 if an odd number of 1s in the data bits is received. Otherwise, the parity bit should be 0. For odd parity, the parity bit should be 1 if an even number of 1s in the data bits is received. Otherwise, the parity bit should be 0.
FE	3	Read only	Framing error. FE is reset to logic 0 whenever SIN is sampled high at the center of the first stop bit, regardless to how many stop bits the UART is configured.
BI	4	Read only	Break interrupt. BI is set to logic 1 whenever SIN is low for longer than a whole data frame (the time of start bit + data bits + parity bit + stop bits). If SIN is still low after BI is reset to logic 0 by reading LSR, BI is not set to logic 1 again. Since break is also a framing error, FE is also set to 1 when BI is set.

Table 10: Line Status Register (LSR, Addr=0x14)

Register Name	Bit	Access Mode	Description
THRE	5	Read only	THR empty. THRE is set to logic 1 whenever THR is empty, which indicates that the transmitter is ready to accept new data to transmit. Pin TxRDYn is the complement of this bit.
TEMT	6	Read only	Both THR and TSR are empty. This bit is set to logic 1 when THRE is set to 1 and the last data bit in the TSR is shifted out through SOUT.

The four error flags (OE, PE, FE, and BI) of LSR are reset to logic 0 after a LSR read.

Modem Status Register

Table 11 provides information about the modem status register.

Table 11: Modem Status Register (MSR, Addr=0x18)

Register Name	Bit	Access Mode	Description
DCTS	0	Read only	Delta CTS indicates that the CTS*n* has changed state since the last time it was read by CPU.
DDSR	1	Read only	Delta DSR indicates that the DSR*n* has changed state since the last time it was read by CP.
TERI	2	Read only	Trailing edge of RI indicates that the RI*n* input has changed from a low to a high state.
DDCD	3	Read only	Delta DCD indicates that the DCD*n* input has changed state.
CTS	4	Read only	Clear to sent is the complement of the CTS*n* input.
DSR	5	Read only	Data set ready is the complement of the DSR*n* input.
RI	6	Read only	Ring indicator is the complement of the RI*n* input.
DCD	7	Read only	Data carrier detect is the complement of the DCD*n* input.

Whenever bit 0-3 is set to logic 1, a modem status interrupt is generated if the interrupt is enabled. These four bits are reset to logic 0 whenever CPU reads MSR.

Baud-Rate Divisor Register

The LatticeMico32 UART includes a baud-rate divisor register, detailed in Table 12.

Table 12: Baud-Rate Divisor Register

Register Name	Bit	Access	Description
DIV	15:0	Write read only	The divisor register is used to divide the CLK_I input to generate the baud-rate clock.

The software can change the value of the divider. The value written to the divider is determined by the following equation:

```
divisor_value = int((CLK_I_frequency * 1000 * 1000)/
(desired_baud_rate * 16))
```

The CLK_I frequency is in megahertz.

Here is an example:

```
CLK_I = 25 MHz
divider = (25*1000*1000)/(115200*16) = 13.6 = 13
```

The structure shown in Figure 5 depicts the register map layout for the UART component. The elements are self-explanatory and are based on the register map shown in Table 4. This structure, which is defined in the MicoUART.h header file, enables you to directly access the UART registers, if desired. It is used internally by the device driver for manipulating the UART.

Figure 5: UART Register Map Structure

```
typedef struct st_MicoUart {
   volatile unsigned int rxtx;
   volatile unsigned int ier;
   volatile unsigned int iir;
   volatile unsigned int lcr;
   volatile unsigned int mcr;
   volatile unsigned int lsr;
   volatile unsigned int msr;
   volatile unsigned int div;
}MicoUart_t;
```

Software Support

This section describes the software support provided for the LatticeMico32 UART component. It first describes the basic UART device-driver interface and then describes the UART lookup service and the UART file-mode service.

The supporting routines are meant for use in a single-threaded environment. If you use them in a multi-tasking environment, you must provide re-entrance protections.

Device Driver

The UART device driver interacts directly with the UART instance. This section describes the limitations, usage model, type definitions, structure, and functions of the UART device driver.

Limitations

The UART device driver assumes that the UART component is used as a basic serial transmission/reception device devoid of hardware flow control.

- Software control: any desired software control must be provided by the application.
- Modem functionality: although the UART component has modem functionality, this feature is not supported in the LatticeMico32 UART component.

Usage Model

The UART device driver interacts with the physical UART device to provide a simple and easy-to-use interface. The UART is a serial input/output device that communicates with another UART. Any errors caused by bad data or mismatched parameters between two UART connections do not lead to shutdown of reception or transmission. When these errors are reported, the device continues to function. From a usage perspective, therefore, there is no real need to have additional infrastructure to handle these errors other than to ignore known faulty data. While the device driver by itself does not provide for error detection or correction or flow control of the streaming data, a higher-level software protocol can provide these facilities.

The device driver presents the UART as a simple character device from which you can either “get” characters or “put” characters for transmission.

In addition, you can configure the UART to operate in an interrupt-driven mode or in a polled mode, configurable through the Mico System Builder (MSB). The operation of either mode is automatic, and you do not need to be concerned with it. The interrupt-driven mode relies on receive and transmit buffers that are implemented in the software. The size of these buffers is configurable through the MSB.

Interrupt-driven mode is more suitable to an asynchronous mode of operation, where the foreground thread of operation does not need to be synchronized to the transmission and reception of data through the UART.

The space required for the buffers, which is a minimum of 2 bytes for transmission and 2 bytes for reception, is always allocated, irrespective of the mode of operation. These allocated buffers are used in interrupt-driven mode but not in polled mode. The compile-time declaration of buffers eliminates the need for performing run-time memory allocation, which allows better memory usage.

Type Definitions

This section describes the type definitions for the UART device context structure.

This structure, shown in Figure 6, contains the UART component instance-specific information and is dynamically generated in the DDStructs.h header file. This information is largely filled in by the managed build process by extracting the UART component-specific information from the platform specification file. You should not manipulate the members directly, because this structure is for exclusive use by the device driver.

Figure 6: UART Device Context Structure

```
typedef struct st_MicoUartCtx_t {
const char* name;
unsigned int base;
unsigned int intrLevel;
unsigned int intrAvail;
unsigned int baudrate;
unsigned int databits;
unsigned int stopbits;
unsigned int rxBufferSize;
unsigned int txBufferSize;
unsigned int blockingTx;
unsigned int blockingRx;
unsigned char *rxBuffer;
unsigned char *txBuffer;
DeviceReg_t lookupReg;
unsigned int rxWriteLoc;
unsigned int rxReadLoc;
unsigned int txWriteLoc;
unsigned int txReadLoc;
unsigned int timeoutMicroSecs;
unsigned int txDataBytes;
unsigned int rxDataBytes;
unsigned int errors;
unsigned int ier;
void * prev;
void * next;
} MicoUartCtx_t;
```

Table 13 describes the parameters of the UART device context structure shown in Figure 6.

Table 13: UART Device Context Structure Parameters

Parameter	Data Type	Description
name	const char *	Instance-specific component name (entered in MSB)
base	unsigned int	MSB-assigned base address for this instance
intrLevel	unsigned int	Processor interrupt line to which this instance is connected
intrAvail	unsigned int	Indicates whether the UART is to be used in interrupt mode (1) or not (0), as configured in MSB
baudrate	unsigned int	Baud rate as configured in MSB. It can be modified at run time.
databits	unsigned int	Data bits as configured in MSB. The data bits can be changed at run time by using the MicoUart_dataConfig function call.
stopbits	unsigned int	Stop bits as configured in MSB. The stop bits can be changed at run time by using the MicoUart_dataConfig function call.
rxBufferSize	unsigned int	Size in bytes of the receive buffer for interrupt-driven operation, as configured in MSB
txBufferSize	unsigned int	Size in bytes of the transmit buffer for interrupt-driven operation, as configured in MSB
blockingTx	unsigned int	Indicates whether the device driver must operate in blocking mode (1) or in non-blocking mode (0) for transmission
blockingRx	unsigned int	Indicates whether the device driver must operate in blocking mode (1) or in non-blocking mode (0) for reception
rxWriteLoc	unsigned int	Used internally for receive buffer management
rxReadLoc	unsigned int	Used internally for receive buffer management
txWriteLoc	unsigned int	Used internally for transmit buffer management
txReadLoc	unsigned int	Used internally for transmit buffer management
timeoutMicroSecs	unsigned int	Used internally to detect device errors on transmission
txDataBytes	unsigned int	Used internally for transmit buffer management

Table 13: UART Device Context Structure Parameters

Parameter	Data Type	Description
rxDataBytes	unsigned int	Used internally for receive buffer management
errors	unsigned int	Not used (reserved for future use)
ier	unsigned int	Used internally as a shadow of the UART IER register
prev	void *	Used internally for device lookup service
next	void *	Used internally for device lookup service
rxBuffer[4]	unsigned char	Internal use for receive buffer implementation. The size of the array is configurable on a per-instance basis through MSB.
txBuffer[4]	unsigned char	Internal use for transmit buffer implementation. The size of the array is configurable on a per-instance basis through MSB.
lookupReg	DeviceReg_t	Used by the device driver to register the UART component instance with the LatticeMico32 lookup service. Refer to the *Software Developer User's Guide* for a description of the DeviceReg_t data type.

Functions

This section describes the implemented device-driver-specific functions.

MicoUartInit Function

```
void MicoUartInit(MicoUartCtx *ctx);
```

This function initializes a LatticeMico32 UART instance on the basis of the passed UART context structure. This initialization function is responsible for the following:

- Initializing the interrupts or buffer parameters for interrupt-driven operation
- Registering the UART instance with UART service

As a part of the managed build process, the LatticeDDInit function calls this initialization routine for each UART instance that is present in the platform.

Table 14 describes the parameter in the MicoUartInit function syntax.

Table 14: MicoUartInit Function Parameter

Parameter	Description
MicoUartCtx_t*	Pointer to a valid MicoUartCtx_t structure representing a valid UART instance.

MicoUart_setRate Function

```
unsigned int MicoUart_setRate(MicoUartCtx_t *ctx, unsigned int baudrate);
```

This function sets the baud rate of a UART instance. The software does not check for the validity of the baud-rate value. The function computes the UART divisor value on the basis of the following formula:

```
divisor_value = int((CLK_I_frequency * 1000 * 1000)/(desired_baud_rate * 16))
```

This result is then loaded into the divisor register, and it also programs the desired baud rate in the UART control register.

Table 15 describes the parameters in the MicoUart_setRate function syntax.

Table 15: MicoUart_setRate Function Parameters

Parameter	Description	Notes
MicoUartCtx_t*	Pointer to a valid MicoUartCtx_t structure representing a valid UART instance.	
unsigned int	Baud rate	No software check is performed. Depending on the CPU clock speed, the baud rate might result in loss of communication or excessive errors. Consult the component data sheet for more information.

Table 16 shows the values returned by the MicoUart_setRate function.

Table 16: Values Returned by the MicoUart_setRate Function

Return Value	Description
0	Successful
MICOUART_ERR_INVALID_ARGUMENT	Failure: baud rate value was zero.

MicoUart_dataConfig Function

```
unsigned int MicoUart_dataConfig(MicoUartCtx_t *ctx,
                                 unsigned int dwidth,
                                 unsigned char parity_en,
                                 unsigned char even_odd,
                                 unsigned int stopbits)
```

This function changes the following UART data-reception transmission parameters:

- Data width
- Enable parity generation and detection
- Parity selection if parity generation and detection is enabled
- Stop bits

Table 17 describes the parameters in the MicoUart_dataConfig function syntax.

Table 17: MicoUart_dataConfig Function Parameters

Parameter	Description
MicoUartCtx_t *ctx	Pointer to a valid MicoUartCtx_t structure representing a valid UART instance
unsigned int dwidth	Data width. Allowed values: 5, 6, 7, 8
unsigned int parity_en	Enable parity ◆ 0 = Parity disabled ◆ 1 = Parity enabled
unsigned int even_odd	Parity type: ◆ 0 = Odd parity ◆ 1 = Even parity **Note:** This parameter is ignored if parity is disabled.
unsigned int stopbits	Stop-bits selection: ◆ 1 = 1 stop bit ◆ 2 = 1.5 or 2 stop bits, depending on the data width

Table 18 shows the values returned by the MicoUart_dataConfig function syntax.

Table 18: Values Returned by the MicoUart_dataConfig Function

Return Value	Description
0	Successful
MICOUART_ERR_INVALID_ARGUMENT	Failure: bad argument to the function.

MicoUart_setBlockMode Function

```
unsigned int MicoUart_setBlockMode(MicoUartCtx_t *ctx, unsigned int uiBlock);
```

This function changes the device driver mode of operation to blocking mode if it was operating in non-blocking mode. Blocking and non-blocking behavior is discussed in “MicoUart_putC Function” on page 23 and “MicoUart_getC Function” on page 24.

Table 19 describes the parameters in the MicoUart_setBlockMode function syntax.

Table 19: MicoUart_setBlockMode Function Parameters

Parameter	Description
MicoUartCtx_t*	Pointer to a valid MicoUartCtx_t structure representing a valid UART instance
unsigned int	0 – Set operation to non-blocking 1 – Set operation to blocking

Table 20 shows the values returned by the MicoUart_setBlockMode function.

Table 20: Value Returned by the MicoUart_setBlockMode Function

Return Value	Description
0	Successful

MicoUart_putC Function

```
unsigned int MicoUart_putC(MicoUartCtx_t *ctx, unsigned char ucChar);
```

This function posts a character to the UART THR for transmission. The behavior depends on the mode of operation:

- Interrupt-driven blocking mode – This function attempts to add the incoming byte of data to the software-implemented transmit buffer. If the buffer is full, the function waits until space is available in the buffer. When space is available, the incoming byte of data is posted to the software-implemented transmit buffer, and the function returns.

- Non-interrupt-driven blocking mode – The function directly polls the UART status register and waits for the UART transmit-hold register to become empty. Once empty, the function queues the character and returns. If the transmit-hold-register is empty, the function loads the character for transmission and returns.
- Interrupt-driven non-blocking mode – The function queries the software-implemented transmit buffer. If there is space in the buffer, it queues the character and returns with a success status. If there is no space in the buffer, it returns the MICOUART_ERR_WOULD_BLOCK error code.
- Non-interrupt driven non-blocking mode – The function queries the UART line status register and checks the THRE bit to see if the THR is empty. If the THR is empty, it is loaded with the incoming data byte, and the function returns. If the UART status register indicates that the transmit-hold register is not empty, the function returns immediately with the MICOUART_ERR_WOULD_BLOCK error code.

Table 21 describes the parameters in the MicoUart_putC function syntax.

Table 21: MicoUart_putC Function Parameters

Parameter	Description
MicoUserCtx_t*	Pointer to a valid MicoUartCtx_t structure representing a valid UART instance
unsigned char	Character to queue for transmission

Table 22 shows the values returned by the MicoUart_putC function.

Table 22: Values Returned by the MicoUart_putC Function

Return Value	Description
0	Successful
MICOUART_ERR_WOULD_BLOCK	UART operates in non-blocking mode, and the function blocks, indicating that there is no room to queue this character.
MICOUART_ERR_DEVICE_ERROR	UART operates in blocking mode, and even after a complete word time, there is no space to queue another character, indicating that this is a device error.

MicoUart_getC Function

```
unsigned int MicoUart_getC(MicoUartCtx_t *ctx, unsigned char
*pucChar);
```

This function attempts to retrieve a received character from the UART. The exact behavior depends on the operating mode:

- Interrupt-driven blocking mode – The function returns immediately with a received character if there is one waiting in the receive buffer. If there is no character pending in the receive buffer, the function continuously polls the

receive buffer for a new character. The function returns when a character is placed into the software-implemented receive buffer.

- Non-interrupt-driven blocking mode – The function returns immediately with the character in the receive-holding register if the UART status indicates there is a character pending. If the UART status indicates there is no character to read, the function continuously polls the UART status register for a new character. The function returns as soon as the RBR is no longer empty.
- Interrupt-driven non-blocking mode – The function returns immediately if there is a new character waiting in the receive buffer. If there is no character waiting in the receive buffer, the function returns immediately with the MICOUART_ERR_WOULD_BLOCK error code.
- Non-interrupt-driven non-blocking mode – The function returns immediately with the character in the UART receive-hold register if the UART status indicates there is a character pending. If there is no character pending, the function returns with the MICOUART_ERR_WOULD_BLOCK error status.

Table 23 describes the parameters in the MicoUart_getC function syntax.

Table 23: MicoUart_getC Function Parameters

Parameter	Description
MicoUartCtx_t*	Pointer to a valid MicoUartCtx_t structure representing a valid UART instance
unsigned char*	Pointer to unsigned character location where the read character should be stored

Table 24 shows the values returned by the MicoUart_getC function.

Table 24: Values Returned by the MicoUart_getC Function

Return Value	Description
0	Successful
MICOUART_ERR_WOULD_BLOCK	UART is operating in non-blocking mode, and the function would block, indicating that there is no character to be read.

Services

The UART component software support includes support for the following two types of services:

- Lookup service – UART device driver registers UART instances with LatticeMico32 lookup service, using their instance names for device names and “UARTDevice” as the device type.

  For information on the LatticeMico32 lookup service, refer to the *LatticeMico32 Software Developer User's Guide*.

- File service – Each UART instance makes itself available as a file device to the LatticeMico32 file services. File device support for the UART is limited to the standard input and output of characters and does not provide an actual file system support. The UART file service has the following limitations:
  - The UART file service does not support an explicit file system, although it does support standard file operations, such as fopen, fprintf, fgets, and fscanf.
  - The UART file system assumes that there is an appropriate connection to the LatticeMico32 UART instance that provides a console for it to communicate with, for example, the LatticeMico32 UART instance connected over a serial link to a PC's serial port, with the PC hosting a serial terminal program such as Hyperlink or TeraTerm.
  - For file-read operations such as fscanf and fgets, the UART file service implementation echoes the characters over the transmit channel. This behavior is not modifiable.
  - The UART file service read function implementation treats a \r or a \n as an input delimiter character. This behavior is not modifiable.
  - The UART file service allows the UART to be treated as a standard I/O stream, provided that it is explicitly set to be used this way through the C/C++ SPE platform configuration tab or at run time programmatically.
  - The UART file system support is limited to single-threaded applications.
  - The UART file system sets the UART to operate in blocking mode. This behavior cannot be modified.

For information on LatticeMico32 file services, refer to the *LatticeMico32 Software Developer User's Guide*.

UART Code Reduction

You can reduce the impact of the UART device driver and services on the overall code size by following these guidelines:

- If appropriate, reduce the transmit and receive buffer sizes to the minimum. You can do this through the MSB user interface.
- If not required, disable the UART file services by defining the following macro in the C/C++ Software Project Environment (SPE) LatticeMico32 Preprocessor defines:

  ```
  _MICOUART_FILESUPPORT_DISABLED_
  ```

  This macro disables the UART-specific file system registration function, allowing the linker to eliminate all the UART file service software code. However, you can no longer use any UART instance for standard input/output or file operation.
- If none of the UART instances operate using interrupts, you can eliminate the code for interrupt-driven operation by defining the _MICOUART_NO_INTERRUPTS_ preprocessor macro. This change

reduces the code size, but the device driver operates on all UART instances present in the system in poll mode.

Software Usage Examples

This section provides examples of using the UART as a file device and as a standard I/O device at run time.

Using a UART as a File Device

The default managed build process initializes the UART instance by invoking the UART initialization function described previously in this document. This initialization function makes the UART available to the LatticeMico32 lookup service, as well as to the LatticeMico32 file service.

Because the UART instance is available as a file device, it can be used for any other file operation, as shown in the example in Figure 7. In the example, it is assumed that the platform contains a UART named “uart.”

Figure 7: UART Used as File Device

```
#include <stdio.h>

int main(void) {

   /*
    * You have a UART instance named "uart" in your platform
    * and you wish to use it as a file device.
    */

   /*
     * STEP 1: "Open" the UART device, using the device-
     * naming convention.
*/

   FILE *fptr = fopen("\\\\uart\\", "wt");
   if(fptr == 0) {
      /* ERROR: failed to open our UART device */
      return(-1);
   }

   /* STEP 2: "Write" some data to our UART device */
   fprintf(fptr, "i wrote bytes to the uart\r\n");
   /* STEP 3: "Close" our device now that we’re done using it.
   */
   fclose(fptr);

   /*
    * Wait for a second just in case the isr is still servicing
    * the transmit buffer, since the UART baud rate is much
    * slower than the CPU clock speed.
   */

   MicoSleepMilliSecs(1000);
   return(0);
}
```

Setting UART as a Standard I/O Device at Run Time

The UART implementation allows C/C++ SPE to flag it as a device capable of serving standard input/output streams. You can also enable this functionality at run time. The example shown in Figure 8 shows how to set a UART instance as a standard input/output device and also demonstrates the lookup capability. The example assumes the presence of a UART component named “uart.”

Figure 8: UART Used as Standard I/O Device

```
#include <stdio.h>
#include "DDStructs.h"
#include "LookupServices.h"
#include "MicoFileDevices.h"
#include "MicoUtils.h"


int main(void) {
   int i;
   char cBuffer[256];


   /*
    * You have a UART instance named "uart" in your platform
    * and you wish to use it as a standard I/O device for
    * printf and fgets (and scanf, etc.)
   */

   /* STEP1: Fetch UART context */
   MicoUartCtx_t *pUart = (MicoUartCtx_t *)
   MicoGetDevice("uart");
   if(pUart == 0) {
      /* ERROR: Failed to find a named device */
      return(-1);
   }

   /* Set this device as standard input device. Device "0" is
   the stdin file handle.*/
   i = MicoFileRedirIO(0, pUart->name);
   if(i != 0) {
      /* ERROR: Failed to set this device for std input */
      return(-1);
   }

   /* Set this device as standard output device. Device "1" is
   the stdout file handle.*/
   i = MicoFileRedirIO(1, pUart->name);
   if(i != 0) {
      /* ERROR: failed to set this device for std output */
      return(-1);
   }

   /*
    * Read a line from the UART!
    * NOTE: Do not enter more than 255 characters!
    */
   gets(cBuffer);
```

Figure 8: UART Used as Standard I/O Device (Continued)

```
    /*
     * "Write" over the UART as standard output!
     */
    printf("You just entered a line using UART: %s\r\n",
    cBuffer);

    /*
     * Wait for a second just in case the isr is still servicing
     * the transmit buffer, since the uart buad rate is much
     * slower than the CPU clock speed
     */
    MicoSleepMilliSecs(1000);

    return(0);
}
```

Using the Device Driver API Directly

The “UartEcho” template provided with the LatticeMico32 System software demonstrates the use of the raw device driver API for manipulating the UART device. This template is located in the following directory:

<*install_path*>\micosystem\utilities\templates\Uart_Echo